

Competitive Programming

Section 33: Inclusion-Exclusion

Competitive Programming Course

2026-07-10

Table of contents

1	The counting problem behind inclusion-exclusion	3
2	Why the alternating signs count each object once	4
3	Diagnosis before formula	5
4	Pattern 1: union of divisibility sets	6
4.1	LCM guard	6
4.2	Worked trace: $N=20$, divisors 2,3,5	6
5	Pattern 2: every symbol appears	7
5.1	Worked trace: length 3, alphabet {a,b,c}	7
6	Pattern 3: bounded stars and bars	8
6.1	Worked trace: three children, four candies, bound two	8
7	Modular arithmetic contract	9
8	Formative exercises	10
9	Exercises	11
10	Local workflow and checklist	12

1 The counting problem behind inclusion-exclusion

Inclusion-exclusion is for a very specific situation: an object can satisfy several conditions, and adding the condition counts counts an object more than once. First write down the *events* or *bad sets*. Only then write a formula.

For two sets,

$$|A \text{ union } B| = |A| + |B| - |A \text{ intersection } B|$$

The intersection was counted twice by the first two terms, so it is removed once. For three sets, add the three singles, subtract the three pairwise intersections, then add the triple intersection. The pattern is:

$$\begin{aligned} &|E_1 \text{ union } \dots \text{ union } E_m| \\ &= \text{sum over nonempty subsets } S \text{ of } \{1, \dots, m\} \\ &\quad (-1)^{|S|+1} * |\text{intersection of } E_i \text{ for } i \text{ in } S| \end{aligned}$$

Odd-size subsets are added; even-size subsets are subtracted.

2 Why the alternating signs count each object once

Take one object that belongs to exactly $t \geq 1$ events. It occurs in $C(t, j)$ intersections of size j , so its total coefficient is

$$C(t, 1) - C(t, 2) + C(t, 3) - \dots + (-1)^{t+1} C(t, t).$$

From the binomial theorem,

$$(1 - 1)^t = C(t, 0) - C(t, 1) + C(t, 2) - \dots + (-1)^t C(t, t) = 0.$$

Move $C(t, 0)=1$ to the other side: the alternating coefficient is exactly 1. An object in no event occurs in no term, so it contributes 0. This is the reusable proof, not a fact to memorize.

3 Diagnosis before formula

Statement shape	Events to define	Intersection count
at least one listed divisor divides x	$d[i]$ divides x	multiples of an LCM
every alphabet symbol must occur	symbol i is missing	strings over remaining symbols
each box has an upper bound	box i exceeds its bound	shifted stars and bars

Ask these four questions before coding:

1. Am I counting a union directly, or subtracting a union of bad events?
2. What does one selected subset of events mean in the original story?
3. Can I count that intersection exactly?
4. Is the number of subsets or terms within the constraints?

4 Pattern 1: union of divisibility sets

For $1 \leq x \leq N$, let event E_i mean that $d[i]$ divides x . If a selected subset has LCM L , its intersection contains exactly $\text{floor}(N/L)$ numbers.

```
count = sum(nonempty subset S) sign(S) * floor(N / lcm(S))
```

This is practical only because problem A has $m \leq 20$: 2^{20} is about one million. Do **not** enumerate subsets when $m=50$ just because the formula is valid.

4.1 LCM guard

The mathematical update is:

```
lcm(a,b) = a / gcd(a,b) * b.
```

Here N and the divisors can reach 10^{18} . We only care whether $L \leq N$. Before multiplying, test whether $a / \text{gcd}(a,b) > N / b$; if so, cap the LCM at $N+1$ and stop that subset. This prevents overflow and records that its contribution is zero.

If the input may contain duplicate divisors, remove duplicates first. The local statement permits them, and treating equal divisors as different events makes the same condition appear twice in inclusion-exclusion.

4.2 Worked trace: $N=20$, divisors 2,3,5

selected set	LCM	$\text{floor}(20 / \text{LCM})$	sign	running total
{2}	2	10	+	10
{3}	3	6	+	16
{5}	5	4	+	20
{2,3}	6	3	-	17
{2,5}	10	2	-	15
{3,5}	15	1	-	14
{2,3,5}	30	0	+	14

The final answer is 14. For a number such as 6, the terms {2}, {3}, and {2,3} give $+1 +1 -1 = 1$, exactly as the proof promised.

5 Pattern 2: every symbol appears

There are k^n strings of length n over k symbols. Let E_i mean that symbol i is missing. If j particular symbols are missing, each position has only $k-j$ choices, hence $(k-j)^n$ strings. There are $C(k, j)$ choices of the missing symbols:

```
answer = sum(j = 0..k) (-1)^j C(k,j) (k-j)^n.
```

The sign starts positive because we begin with all strings and subtract bad strings. In problem B, $k \leq 500$ and $q \leq 200$, so computing each power by binary exponentiation gives $O(q * k * \log n)$, which is intended.

5.1 Worked trace: length 3, alphabet {a,b,c}

```
j = 0: C(3,0) * 3^3 = 27      add  -> 27
j = 1: C(3,1) * 2^3 = 24      subtract -> 3
j = 2: C(3,2) * 1^3 = 3       add   -> 6
j = 3: C(3,3) * 0^3 = 0       subtract -> 6
```

The six strings are the permutations of `abc`. When $n < k$, the same formula evaluates to zero; that is a valuable test, not a special branch you need to invent. For the local $n=0, k=1$ case, the empty string misses the only symbol, so the answer is zero. In code, define `pow(0,0)=1`, as standard binary exponentiation does.

6 Pattern 3: bounded stars and bars

We need nonnegative solutions of

$$x_1 + x_2 + \dots + x_n = s, \quad 0 \leq x_i \leq b.$$

Without upper bounds, Section 32 gives $C(s+n-1, n-1)$. Let E_i be the bad event $x_i \geq b+1$. If a chosen set of j children is bad, set

$$y_i = x_i - (b+1)$$

for those children. This is a bijection with nonnegative solutions whose remaining sum is $s-j(b+1)$. Thus that intersection has

$$C(s-j(b+1)+n-1, n-1)$$

solutions when the remaining sum is nonnegative. Choose the bad children in $C(n, j)$ ways and alternate signs:

$$\text{answer} = \sum_{j \geq 0} (-1)^j C(n, j) C(s-j(b+1)+n-1, n-1).$$

Stop once $j(b+1) > s$: later intersection counts are zero. The course problem also guarantees that the total number of nonzero terms across every query is at most 10^6 ; use that promise when justifying the loop.

6.1 Worked trace: three children, four candies, bound two

First ignore bounds: $C(6, 2)=15$. A selected child who has at least three candies receives three first, leaving one candy to distribute:

```
j=0: + C(3,0) C(6,2) = +15
j=1: - C(3,1) C(3,2) = - 9
j=2: remaining 4-2*3 < 0, stop
answer = 6.
```

For example, $(3, 1, 0)$ is counted in the unrestricted term and once in the bad event for child one, so it cancels. A valid distribution such as $(2, 1, 1)$ is in no bad event and remains once. This is the same object-by-object logic as the general proof.

7 Modular arithmetic contract

Problems B and C use $\text{MOD} = 1,000,000,007$, which is prime. Their factorial tables only contain indices below MOD because the local bounds are at most 10,000. Therefore every factorial denominator is invertible. This method is not a general method for $C(n,k) \bmod \text{MOD}$ when a needed index reaches MOD ; Lucas-style methods are a later topic.

Normalize subtraction in C++:

```
ans = (ans - term + MOD) % MOD;
```

Python can accumulate signed integers and apply $\% \text{MOD}$ at the end of a query.

8 Formative exercises

1. For $N=30$ and divisors 2,6, what should a deduplicated/divisibility-aware inclusion-exclusion computation return? Explain why the intersection is not counted as if the divisors were disjoint.
2. Count length-2 strings over three symbols in which every symbol occurs. Write the four inclusion-exclusion terms before evaluating them.
3. For $x_1+x_2=3$ with $0 \leq x_i \leq 1$, write the $j=0$, $j=1$, and $j=2$ terms. Which terms are zero, and why?
4. A loop has reached an LCM larger than N . Why may it stop building this subset even when more divisors remain?

Answers:

1. 15. Multiples of 6 are already multiples of 2, so $15 + 5 - 5 = 15$.
2. $3^2 - C(3,1)2^2 + C(3,2)1^2 - C(3,3)0^2 = 0$.
3. $C(4,1) - C(2,1)C(2,1) + 0 = 4-4=0$. No distribution fits.
4. Every later LCM is a multiple of the current one, hence also exceeds N and has zero multiples in $[1,N]$.

9 Exercises

Complete the four formative exercises above, then solve A–C in that order: LCM intersections, missing-symbol events, and shifted bounded distributions.

10 Local workflow and checklist

Run the judged local suite with:

```
CP_TARGET=solution python3 sections/33_inclusion_exclusion/check.py
```

Before submitting, check:

```
events are defined in one sentence
the intersection count matches a selected subset, not one event
the sign is determined by the number of selected events
LCM is capped before overflow and duplicate divisors are removed
factorials cover the largest needed index for every query
negative remainders are normalized in C++
```